SIMATS ENGINEERING
Department of Computer Science and Engineering
CSA16 – Data Warehousing and Data Mining
ASSIGNMENT 1 – CO1

Design, Build and Validate a Data Warehouse and OLAP Reporting Solution for a Retail Chain

CO1: Model the data for a data warehouse and apply the various operations that can be performed on the data sets.

Student Name	Register Number	Department / Section	Academic Year / Semester	Faculty Name	Nature of Assignment	Duration	Total Marks	Date of Submission	Date of Evaluation
Challa Jagadeesh	192525403	AI-ML	2026–2027	Dr.D. Beulah David	Individual	1 Week	50	06-11-26	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks / Evidence Expected
1	Decision-Support Requirement	4		At least eight strategic/tactical business questions; measures and dimensions; FreshCart-specific OLTP vs DSS justification.
2	Warehouse Architecture	3		Labelled architecture showing sources, staging, ETL, warehouse, metadata, data marts, OLAP and reporting tools; justified architecture choice.
3	Star Schema Design	6		Declared grain; fact measures and additivity; dimensions and hierarchies; keys; conformed Returns fact; schema diagram and DDL.
4	Snowflake Schema Comparison	4		Product and Store snowflake designs; storage calculation; join counts for three queries; performance, ETL and usability comparison; justified choice.
5	ETL Process Design	5		Data-quality issues and transformations; surrogate-key lookup; unknown/late dimensions; SCD choice; load sequence; rejected rows/audit; before-and-after extract.
6	Data Cube and OLAP Operations	5		Cuboid calculations with/without hierarchies; base/apex cuboids; roll-up, drill-down, slice, dice and pivot with FreshCart questions and SQL; materialisation choice.
7	OLAP Application Development	5		Working interactive OLAP application; documented data generation and scale; all five OLAP operations; screenshots, source code and reconciliation.
8	Query Processing Performance	4		Three queries tested on OLTP, star schema and indexed/summary design; execution time, rows read, joins, query plans, storage/load cost and interpretation.
9	KNIME Reporting Workflow	3		Annotated KNIME workflow; node purposes and ETL/OLAP mapping; exported report; limitation at 1,000,000+ rows.
10	Final Design Recommendation	3		Evidence-based recommendation of architecture/schema; comparison with alternatives; trade-offs, load cost, limitations and scalability.
11A	Design Challenge – Impact Analysis	2		Identifies affected and unaffected schema, ETL, cube and architecture components; distinguishes structural, loading and performance impacts.
11B	Design Challenge – Evaluate Original Design	2		Tests original design against 20,000 daily online orders and hourly refresh; identifies adequate areas and limitations.
11C	Design Challenge – Adaptation or Redesign	2		Presents revised schema/ETL flow, product-code mapping, delivery-partner attribute, SCD treatment and incremental/micro-batch loading or aggregates.
11D	Design Challenge – Evidence-Based Defence	2		Supports decision with measured/calculated load time, query time and storage; identifies deployment monitoring metrics.
	TOTAL	**50**		**Marks to be awarded by the faculty.**

OVERALL REMARKS

Faculty feedback:

_____

_____

Student Signature	Faculty Signature	Program Director
		_

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present P = partly present ✗ = missing. Mark the status after checking the student's submitted report and supporting evidence.

Task	Evidence Expected	Status	Note
1	Business questions, measures/dimensions and OLTP–DSS analysis	☐	At least eight questions; FreshCart-specific explanation.
2	Warehouse architecture diagram and justification	☐	All required layers, tools and architecture type shown.
3	Star schema, grain, measures, dimensions, keys and DDL	☐	Sales and Returns facts; conformed dimensions demonstrated.
4	Snowflake schema and quantitative comparison	☐	Storage calculations, join counts and design decision included.
5	ETL flow, cleaning rules, SCD decision and cleaned data	☐	All sample defects addressed; rejected rows and audit included.
6	Cube lattice calculations and five OLAP operations	☐	Business questions, cuboids and equivalent SQL included.
7	Working OLAP application and validation evidence	☐	Data sizes, user-selected operations, screenshots, code and reconciliation.
8	Query performance results and query plans	☐	Three queries across three designs; time, rows, joins and storage/load cost.
9	KNIME workflow and exported management report	☐	Annotated nodes and ETL/OLAP concept mapping; limitation discussed.
10	Final design recommendation	☐	Supported by the student's comparisons and measured results.
11A	Impact analysis of changed requirements	☐	Affected and unaffected components identified.
11B	Evaluation under new online-channel workload	☐	20,000 orders/day and hourly refresh considered/tested.
11C	Revised design or adaptation	☐	New attribute, code mapping, SCD and load strategy addressed.
11D	Evidence-based defence and monitoring plan	☐	Measurements/calculations and monitoring metrics included.

IMPLEMENTATION EVIDENCE CHAIN (Tasks 7–9)

Evidence Item	Status	Observation
Warehouse schema and DDL	☐	Star/snowflake structures, keys, constraints and declared grain are documented.
Data source and generation	☐	Real or synthetic data source and generation method documented for 1,000 to 1,000,000+ fact rows.
Application source code	☐	Readable OLAP application code, language, dependencies and execution instructions provided.
Interactive OLAP inputs	☐	User can select operation, dimension, hierarchy level and filter values.
Expected and actual results	☐	Outputs for roll-up, drill-down, slice, dice and pivot are recorded and interpreted.
Reconciliation evidence	☐	Independent totals agree with source data or differences are explained.
Performance test evidence	☐	Queries, data sizes, execution times, rows read, joins and query plans recorded.
Index / summary structure evidence	☐	Additional storage and load cost are recorded and trade-offs explained.
KNIME workflow evidence	☐	Workflow screenshot, node descriptions and exported report included.
Reproducibility and consistency	☐	Code, data, screenshots, tables and report values are consistent and reproducible.

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All Tasks 1–10 and Day-4 Design Challenge 11A–11D answered	☐ Yes ☐ Partly ☐ No
Business questions, architecture and dimensional model are FreshCart-specific	☐ Yes ☐ Partly ☐ No
Fact-table grain, measures, dimensions and keys are clearly stated	☐ Yes ☐ Partly ☐ No
Star and snowflake designs are compared using calculations and examples	☐ Yes ☐ Partly ☐ No
ETL transformations, SCD handling and cleaned sample data are shown	☐ Yes ☐ Partly ☐ No
All five OLAP operations are demonstrated with relevant queries	☐ Yes ☐ Partly ☐ No
OLAP application, source code, screenshots and reconciliation are provided	☐ Yes ☐ Partly ☐ No
Query performance tests include plans, rows read and storage/load costs	☐ Yes ☐ Partly ☐ No
KNIME workflow and exported report are included	☐ Yes ☐ Partly ☐ No
Day-4 challenge addresses online orders, hourly refresh and changing customer details	☐ Yes ☐ Partly ☐ No
All diagrams, tables, calculations and references are numbered and readable	☐ Yes ☐ Partly ☐ No
No unfilled template placeholders remain in the student report	☐ Yes ☐ Partly ☐ No
Viva recommended where implementation, calculations or results need clarification	☐ Yes ☐ No

Note: This DWDM assignment evaluation template is based on Assignment 1 – CO1. Blank marks and evidence statuses are intentionally left for faculty evaluation.

SIMATS ENGINEERING

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
CSA16 Data Warehousing and Data Mining
Assignment 1 - CO1

CO1 Coverage: *Operational vs decision-support systems, data warehouse architecture, ETL, dimensional data modelling (star and snowflake schemas), data cubes, OLAP operations, query processing, open-source BI tools and KNIME for reporting.*

Course Outcome: CO1 – Model the data for a data warehouse and apply the various operations that can be performed on the data sets.

Assignment Title: Design, Build and Validate a Data Warehouse and OLAP Reporting Solution for a Retail Chain (Total Marks - 50)

Duration: 1 week

Scenario — FreshCart Retail Analytics

FreshCart operates 120 supermarkets across six states. Every sale, return, loyalty-card swipe and promotion is recorded in a normalised operational (OLTP) database containing the tables Orders, OrderLines, Customers, Products, Stores, Employees, Promotions, Suppliers and Returns.

Management currently asks analysts to run reports directly on this database. Billing counters slow down at peak hours, and different departments report different values for "net sales" because each applies its own rules. Management has asked you to design a **data warehouse** that supports questions such as:

- Which product categories show declining net sales, quarter over quarter, in each region?
- Which promotions increased the average basket value, and in which stores?
- How does weekend sales performance differ from weekday performance for each department?
- Which customer segments return the most items, and of which products?
- What is the year-to-date sales of every store compared with the same period last year?

The sales fact data may contain:

1,000 → 10,000 → 100,000 → 1,000,000+ sales line items

You are required to investigate, design and compare alternative warehouse designs, and to demonstrate OLAP operations on the design you recommend. Your solution must remain practical as data grows. Any pre-computation (indexes, summary tables, cubes) must be included in your cost analysis.

Design Challenge:

FreshCart now launches an online channel that sends about 20,000 orders per day from a new source system. Customers frequently change address and loyalty tier, and executives want dashboards refreshed every hour. Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Analyse the Decision-Support Requirement **4 Marks**

Examine why FreshCart's existing operational system is unsuitable for management analysis, and define what the warehouse must support.

Discuss:

- At least **eight business questions** (including the five above) classified as strategic or tactical
- For each question, the **measures** and **dimensions** it requires
- How the operational system and a decision-support system differ in purpose, data currency, structure, query pattern, users and volume
- Which of your questions cannot be answered efficiently from the OLTP database, and why

Do not simply provide a table of textbook differences. You are expected to **relate each difference to FreshCart's situation**.

Evidence Expected
Business questions with their measures and dimensions, and a justified explanation of why a separate decision-support system is required for FreshCart.

Answer:

1.1 Business Questions, Measures and Dimensions

FreshCart requires a Data Warehouse to analyse sales, customer behaviour, promotions and store performance. The following business questions are classified as strategic or tactical.

Business Question	Type	Measures	Dimensions
Which product categories show declining net sales quarter over quarter in each region?	Strategic	Net Sales, QoQ Change %	Time, Product, Store
Which promotions increased average basket value, and in which stores?	Tactical	Sales Amount, Order Count, Average Basket Value	Promotion, Store, Date
How does weekend sales performance differ from weekday performance for each department?	Tactical	Net Sales, Quantity Sold	Date, Product, Store
Which customer segments return the most items, and of which products?	Strategic	Return Quantity, Return Value, Return Rate	Customer, Product, Date
What is the year-to-date sales of every store compared with the same period last year?	Strategic	YTD Sales, YoY Difference	Date, Store
Which stores have the highest average transaction value this month?	Tactical	Net Sales, Number of Orders, Average Transaction Value	Store, Date
Which regions contribute the highest revenue annually?	Strategic	Net Sales, Revenue Share	Store, Date
Which product categories have the highest return rate?	Tactical	Return Quantity, Return Rate	Product, Date, Store

1.2 Difference Between Operational System and Decision-Support System

Aspect	Operational System (OLTP)	Decision-Support System (DSS)
Purpose	Handles daily transactions such as billing and order processing	Supports business analysis and decision-making
Data Currency	Contains current operational data	Contains historical and integrated data
Structure	Normalized tables to reduce redundancy	Fact tables and dimension tables
Query Pattern	Short transactions, inserts and updates	Complex queries, joins and aggregations
Users	Cashiers, employees and operational staff	Managers, analysts and executives
Data Volume	Handles individual transactions	Handles large historical datasets
Performance	Optimized for transaction speed	Optimized for analytical queries
FreshCart Application	Processes supermarket billing and returns	Analyses sales trends, customer behaviour and regional performance

1.3 Why FreshCart Requires a Separate Data Warehouse

FreshCart's operational database is unsuitable for complex management analysis because analytical queries require joining multiple normalized tables and processing large amounts of historical data.

For example, calculating quarter-over-quarter sales by product category and region requires aggregating sales across several stores and multiple quarters. Such queries may slow down billing counters during peak hours.

A separate Data Warehouse solves these problems by:

- Integrating data from different operational sources.
- Maintaining historical sales and customer information.
- Providing a common definition of Net Sales.
- Supporting complex analytical queries efficiently.
- Improving reporting performance.
- Enabling strategic and tactical decision-making.

Therefore, a separate Decision-Support System is essential for FreshCart to improve operational performance and provide accurate business insights.

TASK 2 – Propose the Warehouse Architecture **3 Marks**

Propose an architecture for FreshCart and represent it using a labelled diagram.

Your architecture must show:

- Data sources, staging area, ETL process, warehouse storage and metadata repository
- Data marts, if any, and the OLAP server

- Front-end tools for reporting, query and analysis
- Your choice among an enterprise warehouse, independent data marts, dependent data marts or a virtual warehouse, and **why**

A copied textbook architecture without justification for FreshCart's needs will **not** receive full credit.

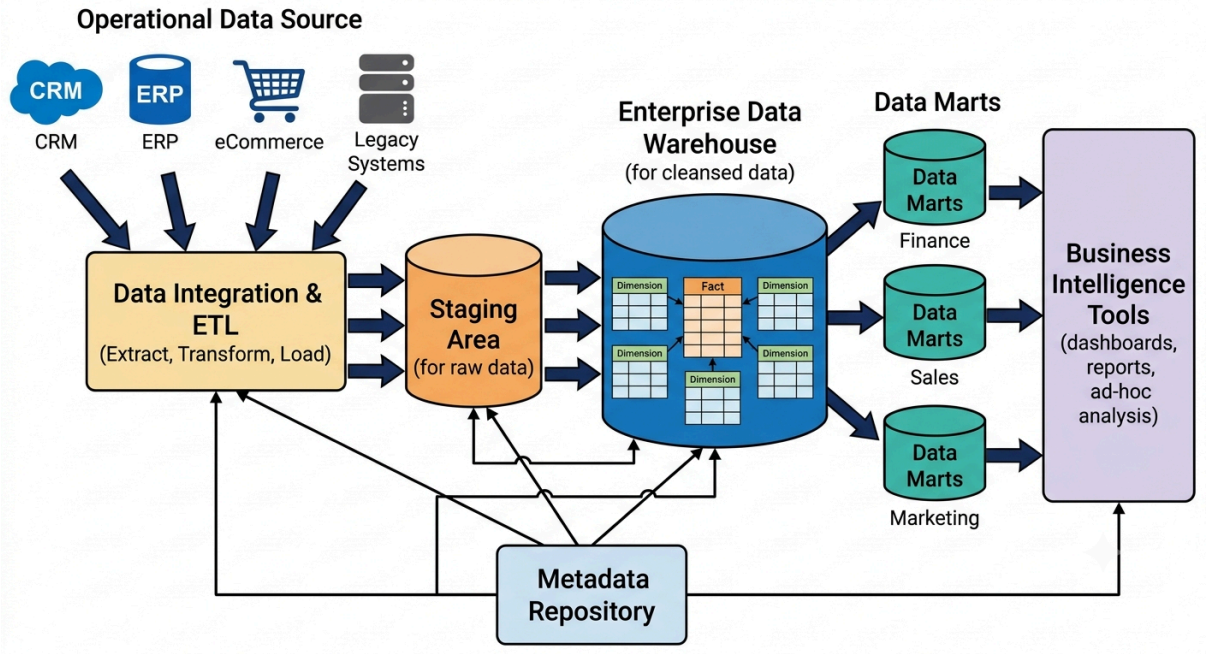
Evidence Expected
A labelled architecture diagram together with a reasoned justification of the selected architecture type.

Answer:

2.1 Proposed Data Warehouse Architecture

FreshCart requires an Enterprise Data Warehouse with dependent data marts to provide integrated and consistent analytical reporting.

Architecture **Diagram**




```
None
OPERATIONAL DATA SOURCES
```

2.2 Architecture Selection and Justification

The recommended architecture is an Enterprise Data Warehouse with Dependent Data Marts.

FreshCart operates 120 supermarkets across six states and requires consistent sales analysis across all departments. A centralized Enterprise Data Warehouse integrates data from all stores and provides a common definition of Net Sales.

Dependent data marts are created from the Enterprise Data Warehouse to support sales and returns analysis.

This architecture is preferred because:

- It provides integrated and consistent data.

- It avoids conflicting reports from different departments.
- It supports historical analysis across stores and regions.
- It improves analytical query performance.
- It supports future expansion and large data volumes.

An independent data-mart architecture may produce inconsistent results, while a virtual warehouse repeatedly queries operational systems and may affect billing performance.

TASK 3 – Design the Star Schema **6 Marks**

Design a dimensional model for FreshCart's sales analysis using a **star schema**.

You must:

- State the **business process** and declare the **grain** of the fact table *before* listing measures
- Identify the fact table and its measures, and classify each as **additive, semi-additive or non-additive**
- Identify the dimension tables (at minimum Date, Product, Store, Customer and Promotion) with their attributes and **concept hierarchies**
- Define primary keys, surrogate keys and foreign keys
- Show how a second fact table (for example Returns) can share **conformed dimensions** with the Sales fact table
- Provide the schema diagram and the table-creation (DDL) statements

You must **declare the grain explicitly** and show that every measure and dimension is consistent with it.

Evidence Expected
A star-schema diagram, declared grain, classified measures, dimension hierarchies, keys and DDL statements consistent with the declared grain.

Answer:

3.1 Business Process and Grain

Business Process: Retail Sales Analysis.

Grain: Each row in the FactSales table represents one product line of an order purchased by one customer at one store on one date, associated with one promotion or no promotion.

All measures and dimensions are defined consistently with this grain.

3.2 Fact Table and Measures

FactSales

Measure	Description	Additivity
Quantity	Number of units sold	Additive
GrossAmount	Quantity × Unit Price	Additive
DiscountAmount	Discount applied	Additive

NetSales	Gross Amount − Discount Amount	Additive
UnitPrice	Price per unit	Non-additive

Average Basket Value is a derived measure calculated as:

Average Basket Value = Total Net Sales / Number of Orders

It should not be directly summed.

3.3 Dimension Tables and Hierarchies

Dimension	Attributes	Hierarchy
DimDate	DateKey, FullDate, Day, DayName, IsWeekend, Month, Quarter, Year	Day → Month → Quarter → Year
DimProduct	ProductKey, ProductCode, ProductName, Brand, Category, Department	Product → Category → Department
DimStore	StoreKey, StoreCode, StoreName, City, State, Region	Store → City → State → Region
DimCustomer	CustomerKey, CustomerID, CustomerName, Segment, LoyaltyTier, Address	Customer → Segment
DimPromotion	PromotionKey, PromotionCode, PromotionName, PromotionType, StartDate, EndDate	Promotion

3.4 Star Schema Diagram

```
None
                  DimDate
                     |
                     |
DimProduct ----- FactSales ----- DimStore
                     |
                     |
                DimCustomer
                     |
                     |
                DimPromotion
```

FactSales contains foreign keys referencing the primary keys of all dimension tables.

FactReturns is another fact table that shares the conformed dimensions DimDate, DimProduct, DimStore, DimCustomer and DimPromotion.

3.5 Primary Keys, Surrogate Keys and Foreign Keys

- Primary Key: Uniquely identifies each record in a table.
- Surrogate Key: Artificial warehouse-generated key, usually an integer.
- Natural Key: Original business identifier from the source system.
- Foreign Key: References the primary key of another table.

Example:

- ProductKey → Surrogate Key
- ProductCode → Natural Key
- FactSales.ProductKey → Foreign Key referencing DimProduct.ProductKey

Unknown dimension values are represented using a predefined Unknown key, such as 0.

3.6 SQL DDL Statements

```SQL
CREATE TABLE DimDate (
    DateKey INT PRIMARY KEY,
    FullDate DATE,
    DayNum INT,
    DayName VARCHAR(10),
    IsWeekend BOOLEAN,
    MonthNum INT,
    QuarterNum INT,
    YearNum INT
);

CREATE TABLE DimProduct (
    ProductKey INT PRIMARY KEY,
    ProductCode VARCHAR(30),
    ProductName VARCHAR(100),
    Brand VARCHAR(60),
    Category VARCHAR(60),
    Department VARCHAR(60)
);

CREATE TABLE DimStore (
```

```
    StoreKey INT PRIMARY KEY,
    StoreCode VARCHAR(30),
    StoreName VARCHAR(80),
    City VARCHAR(50),
    State VARCHAR(50),
    Region VARCHAR(30)
);

CREATE TABLE DimCustomer (
    CustomerKey INT PRIMARY KEY,
    CustomerID VARCHAR(30),
    CustomerName VARCHAR(100),
    Segment VARCHAR(40),
    LoyaltyTier VARCHAR(30),
    Address VARCHAR(200),
    EffectiveFrom DATE,
    EffectiveTo DATE,
    IsCurrent BOOLEAN
);

CREATE TABLE DimPromotion (
    PromotionKey INT PRIMARY KEY,
    PromotionCode VARCHAR(30),
    PromotionName VARCHAR(100),
    PromotionType VARCHAR(40)
);

CREATE TABLE FactSales (
    SalesLineKey BIGINT PRIMARY KEY,
    OrderID VARCHAR(30),
    DateKey INT,
    ProductKey INT,
    StoreKey INT,
    CustomerKey INT,
```

```
    PromotionKey INT,
    Quantity DECIMAL(12,2),
    UnitPrice DECIMAL(12,2),
    GrossAmount DECIMAL(14,2),
    DiscountAmount DECIMAL(14,2),
    NetSales DECIMAL(14,2),

    FOREIGN KEY (DateKey) REFERENCES DimDate(DateKey),
    FOREIGN KEY (ProductKey) REFERENCES DimProduct(ProductKey),
    FOREIGN KEY (StoreKey) REFERENCES DimStore(StoreKey),
    FOREIGN KEY (CustomerKey) REFERENCES DimCustomer(CustomerKey),
    FOREIGN KEY (PromotionKey) REFERENCES DimPromotion(PromotionKey)
);

CREATE TABLE FactReturns (
    ReturnLineKey BIGINT PRIMARY KEY,
    ReturnID VARCHAR(30),
    DateKey INT,
    ProductKey INT,
    StoreKey INT,
    CustomerKey INT,
    PromotionKey INT,
    ReturnedQty DECIMAL(12,2),
    ReturnAmount DECIMAL(14,2),

    FOREIGN KEY (DateKey) REFERENCES DimDate(DateKey),
    FOREIGN KEY (ProductKey) REFERENCES DimProduct(ProductKey),
    FOREIGN KEY (StoreKey) REFERENCES DimStore(StoreKey),
    FOREIGN KEY (CustomerKey) REFERENCES DimCustomer(CustomerKey),
    FOREIGN KEY (PromotionKey) REFERENCES DimPromotion(PromotionKey)
);
```

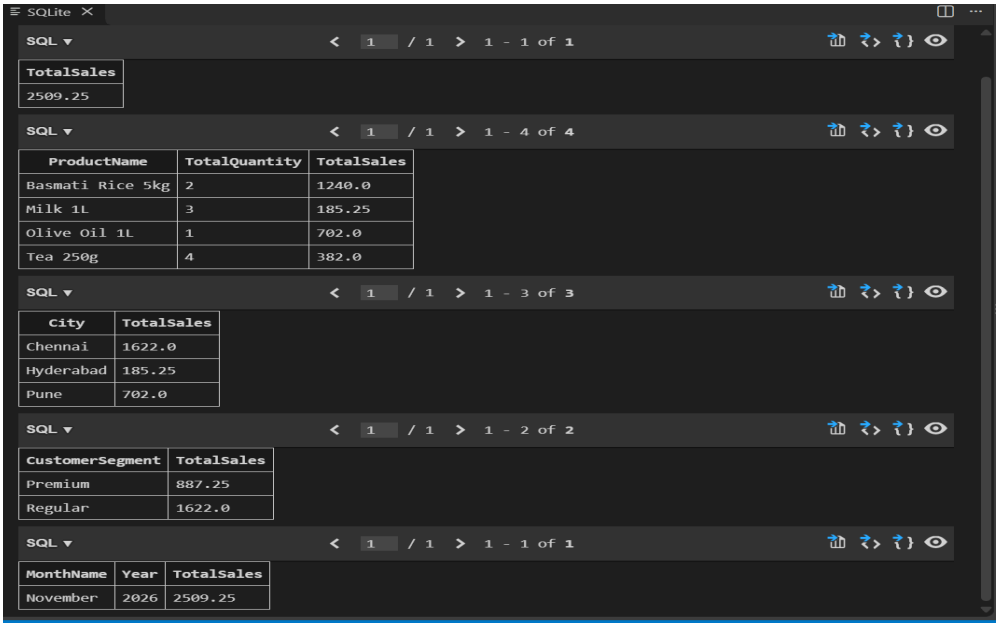
3.7 Second Fact Table

FactReturns stores return transactions separately from sales transactions.

It contains:

- ReturnLineKey
- ReturnID
- DateKey
- ProductKey
- StoreKey
- CustomerKey
- PromotionKey
- ReturnedQty
- ReturnAmount

FactReturns shares conformed dimensions with FactSales, allowing FreshCart to analyse sales and returns using the same product, store, customer and date information.



TASK 4 – Compare with a Snowflake Schema **4 Marks**

Create a snowflaked version of the Product dimension (product → brand → category → department) and the Store dimension (store → city → state → region).

Assume 50,000 products, 800 brands, 60 categories, 12 departments, 120 stores, 70 cities, 6 states and 3 regions.

Discuss:

- The additional tables, keys and joins introduced
- The estimated change in dimension storage (show your calculation)
- The number of joins needed for three representative queries in the star and snowflake versions
- Effect on query performance, ETL maintenance and ease of understanding for business users
- Which design you would choose for FreshCart, and under what condition you would change that decision

You are expected to **support the comparison with calculations and examples**, rather than repeat that “snowflake saves space but is slower”.

Evidence Expected

A snowflake design, storage and join-count calculations, and a justified schema choice.

Answer:

4.1 Snowflake Schema Design

In a Snowflake Schema, dimension tables are normalized into additional tables to reduce redundancy.

Product Snowflake

```
None
DimProduct
(ProductKey, ProductCode, ProductName, BrandKey)
        |
        v
DimBrand
(BrandKey, BrandName, CategoryKey)
        |
        v
DimCategory
(CategoryKey, CategoryName, DepartmentKey)
        |
        v
DimDepartment
(DepartmentKey, DepartmentName)
```

Store Snowflake

```
None
DimStore
(StoreKey, StoreCode, StoreName, CityKey)
        |
        v
DimCity
(CityKey, CityName, StateKey)
        |
        v
DimState
(StateKey, StateName, RegionKey)
        |
        v
DimRegion
```

```
(RegionKey, RegionName)
```

4.2 Dimension Storage Calculation

Assumption: Each repeated descriptive label occupies approximately 20 bytes. This calculation considers label storage only and excludes keys, indexes and row overhead.

Attribute	Star Schema Storage	Snowflake Storage	Savings
Brand	50,000 × 20 = 1,000,000 B	800 × 20 = 16,000 B	984,000 B
Category	50,000 × 20 = 1,000,000 B	60 × 20 = 1,200 B	998,800 B
Department	50,000 × 20 = 1,000,000 B	12 × 20 = 240 B	999,760 B
City	120 × 20 = 2,400 B	70 × 20 = 1,400 B	1,000 B
State	120 × 20 = 2,400 B	6 × 20 = 120 B	2,280 B
Region	120 × 20 = 2,400 B	3 × 20 = 60 B	2,340 B
Total	3,009,600 B	19,020 B	2,990,580 B

Approximate label-storage reduction = 99.37%.

4.3 Join Comparison

Query	Star Schema Joins	Snowflake Schema Joins
Sales by Product Category	1	3
Sales by Department and Region	2	6
Sales by Store City	1	2

4.4 Performance, ETL and Business Usability

Star Schema:

- Requires fewer joins.
- Provides faster analytical query execution in common cases.
- Is easier for business users to understand.
- Simplifies reporting and BI tool integration.
- Contains repeated descriptive attributes.

Snowflake Schema:

- Reduces duplication of descriptive attributes.
- Improves centralized hierarchy maintenance.

- Requires additional joins.
- Increases ETL complexity.
- Makes reporting structures more difficult for business users to understand.

4.5 Recommended Schema

FreshCart should use the Star Schema for its primary analytical warehouse.

Although Snowflake reduces dimension-label redundancy, the Star Schema provides simpler queries, fewer joins and easier reporting for business users.

Snowflake can be considered if dimension redundancy becomes significant or hierarchy attributes require centralized independent maintenance.

TASK 5 – Design the ETL Process **5 Marks**

The following extract from the source systems is to be loaded into the warehouse:

Order ID	Customer	Store	Product	Qty	Unit Price	Order Date
1001	Rajesh Kumar	Chennai-04	Basmati Rice 5kg	2	620.00	03/11/2026
1002	R. Kumar	chennai-04	Basmati rice 5 kg	1	620.00	2026-11-03
1003	Anita Joseph	Pune-11	Olive Oil 1L	-3	780.00	04-Nov-26
1004	Meena S	(null)	Tea 250g	4	95.50	04/11/2026
1005	Anita Joseph	Pune-11	Olive Oil 1L	3	9.40 (USD)	04/11/2026
1006	Vikram Rao	Hyd-02	(null)	1	1,250	05/11/2026

Design the ETL process for loading this data.

Discuss:

- Every data-quality problem in the extract and the **transformation rule** that resolves it
- Surrogate-key lookup and how unknown or late-arriving dimension members are handled
- How a change of customer address will be handled using a **slowly changing dimension** technique (Type 1, 2 or 3) and why
- The difference between the **initial load** and the **incremental load**, and the order in which dimensions and facts are loaded
- Handling of rejected records and audit of row counts

Show the **before and after** version of the extract. Merely listing ETL stage names will not receive full credit.

Evidence Expected

An ETL flow diagram, transformation rules for each defect, SCD decision with justification and the cleaned version of the sample extract.

Answer:

5.1 Data Quality Problems in the Given Dataset

The given FreshCart dataset contains the following data quality issues:

Order ID	Data Quality Problem	ETL Solution
1001	Date format DD/MM/YYYY	Convert to standard YYYY-MM-DD
1002	Customer name abbreviation: R. Kumar	Match using CustomerID and standardize name
1002	Product name has inconsistent spacing and capitalization	Standardize product name
1002	Store code lowercase	Convert to uppercase
1003	Negative quantity (-3)	Identify as return transaction if confirmed; otherwise reject
1004	Missing Store value	Assign Unknown Store key and send for investigation
1005	Unit price in USD	Convert to INR using approved exchange rate
1006	Missing Product value	Assign Unknown Product key and quarantine record
1006	Price contains comma formatting	Convert 1,250 to numeric 1250

5.2 ETL Data Cleaning Rules

The following transformation rules are applied:

1. Standardize all date values into YYYY-MM-DD format.
2. Convert customer names, product names and store codes into consistent formats.
3. Remove unnecessary spaces and normalize capitalization.
4. Match customer records using unique CustomerID rather than names alone.
5. Convert all monetary values into INR using the approved exchange rate.
6. Convert quantity and price columns into numeric data types.
7. Validate quantity and price values before loading.
8. Handle missing dimension values using Unknown dimension records.
9. Identify negative quantities as possible return transactions.
10. Remove duplicate records using OrderID and transaction-line identifiers.
11. Maintain rejected records in a quarantine table for correction.
12. Maintain audit logs containing source count, inserted rows, rejected rows and execution time.

5.3 Cleaned Dataset

Order ID	Customer	Store	Product	Quantity	Unit Price	Order Date
1001	Rajesh Kumar	CHENNAI-04	BASMATI-RICE-5KG	2	620 INR	2026-11-03

1002	Rajesh Kumar*	CHENNAI-04	BASMATI-RICE-5KG	1	620 INR	2026-11-03
1003	Anita Joseph	PUNE-11	OLIVE-OIL-1L	-3	780 INR	2026-11-04
1004	Meena S	UNKNOWN	TEA-250G	4	95.50 INR	2026-11-04
1005	Anita Joseph	PUNE-11	OLIVE-OIL-1L	3	9.40 USD converted to INR	2026-11-04
1006	Vikram Rao	HYD-02	UNKNOWN	1	1250 INR	2026-11-05

Customer name standardization is performed only after confirming the matching CustomerID.

The exchange rate for Order 1005 must be obtained from the approved exchange-rate source. The exact INR value cannot be calculated without that rate.

Order 1003 is classified as a return only after confirmation from the source transaction system.

5.4 ETL Workflow

```
None
Operational Sources
        |
        v
     Extract
        |
        v
   Raw Staging
        |
        v
 Data Validation
        |
        v
 Data Cleaning
        |
        v
 Transformation
        |
```

```
       v
Dimension Lookup
       |
       v
Surrogate Key Assignment
       |
       v
Dimension Loading
       |
       v
Fact Table Loading
       |
       v
Reconciliation
       |
       v
Data Warehouse
```

Invalid records are stored separately in a quarantine table.

5.5 Slowly Changing Dimension (SCD)

FreshCart should use SCD Type 2 for Customer Address and Loyalty Tier changes.

When a customer's address changes:

- The existing customer record is marked as inactive.
- Its EffectiveTo date is updated.
- A new customer dimension record is inserted.
- A new CustomerKey is assigned.
- Historical sales continue referencing the earlier CustomerKey.

This allows FreshCart to analyse customer information as it existed at the time of the transaction.

5.6 Initial and Incremental Loading

Initial Load:

- Extract all available historical data.
- Clean and transform the data.
- Load dimensions first.
- Load fact tables after dimension loading.
- Validate totals and record counts.

Incremental Load:

- Extract only new or modified records using CDC or a watermark.
- Apply transformations and validation.
- Update dimensions according to SCD rules.
- Insert new fact records.
- Maintain audit logs and reconciliation totals.

Incremental loading reduces processing time and avoids repeatedly loading the entire dataset.

```
Detected CSV headers: ['OrderID', 'Customer', 'Store', 'Product', 'Qty', 'UnitPrice', 'OrderDate']

==================================
      FRESHCART ETL SUMMARY
==================================
Source rows : 6
Loaded rows : 4
Rejected    : 2
Skipped     : 0

Row reconciliation:
6 = 4 + 2 + 0

Cleaned CSV : etl_output\cleaned_sales.csv
Rejected CSV: etl_output\rejected_sales.csv

==================================
       REJECT DETAILS
==================================
1005 -> USD price requires approved exchange rate
1003 -> Negative/zero quantity: return review required

==================================
       LOADED SALES
==================================
('1001', 2, 1240.0, 1240.0)
('1002', 1, 620.0, 620.0)
('1004', 4, 382.0, 382.0)
('1006', 1, 1250.0, 1250.0)

==================================
ETL EXECUTION COMPLETED
==================================
```

TASK 6 – Model the Data Cube and Apply OLAP Operations **5 Marks**

FreshCart's analysts view sales as a data cube with these dimensions and hierarchy levels:

- **Time:** day → month → quarter → year
- **Product:** product → category → department
- **Store:** store → city → state
- **Customer segment:** segment (no hierarchy)

Discuss:

- The total number of cuboids in the lattice **with** the hierarchies above, and **without** them (show the calculation)
- The base cuboid and the apex cuboid, and what each contains
- One business question for each of **roll-up, drill-down, slice, dice and pivot**, with the cuboid it uses and an equivalent SQL statement
- Which cuboids you would **materialise** if storage allows only a small fraction of the lattice, and the reasoning for your selection

You are expected to **connect each OLAP operation to a specific FreshCart business question**, rather than define the operations generally.

Evidence Expected
Cuboid calculations, five OLAP operations with queries, and a justified partial-materialisation choice.

Answer:

6.1 Cube Dimensions and Hierarchies

FreshCart Sales Cube contains four dimensions:

Dimension	Hierarchy
Time	Day → Month → Quarter → Year
Product	Product → Category → Department
Store	Store → City → State
Customer	Customer → Segment

The primary measure is NetSales.

6.2 Cube Structure and Number of Cuboids

The hierarchy-aware cube has the following levels:

- Time: 5 levels including ALL
- Product: 4 levels including ALL
- Store: 4 levels including ALL
- Customer: 2 levels including ALL

Total cuboids:

$$5 \times 4 \times 4 \times 2 = 160$$

Therefore, the cube contains 160 hierarchy-aware cuboids.

The base cuboid contains detailed sales records at the lowest dimension levels, while the apex cuboid contains total sales across all dimensions.

Base Cuboid: Day × Product × Store × Customer Segment

Apex Cuboid: ALL × ALL × ALL × ALL

6.3 OLAP Operations

A. Roll-Up

Roll-up aggregates detailed sales data into higher levels of a hierarchy.

Example: Daily sales are aggregated into quarterly sales.

```
None
SELECT
    d.YearNum,
    d.QuarterNum,
    p.Category,
    s.Region,
    SUM(f.NetSales) AS TotalSales
FROM FactSales f
JOIN DimDate d ON f.DateKey = d.DateKey
JOIN DimProduct p ON f.ProductKey = p.ProductKey
JOIN DimStore s ON f.StoreKey = s.StoreKey
GROUP BY
    d.YearNum,
    d.QuarterNum,
    p.Category,
    s.Region;
```

B. Drill-Down

Drill-down moves from summarized data to more detailed information.

Example: Quarterly sales are expanded into monthly sales by store.

```
None
SELECT
    d.MonthNum,
    s.StoreName,
    SUM(f.NetSales) AS TotalSales
FROM FactSales f
JOIN DimDate d ON f.DateKey = d.DateKey
JOIN DimStore s ON f.StoreKey = s.StoreKey
WHERE d.YearNum = 2026
  AND d.QuarterNum = 4
GROUP BY d.MonthNum, s.StoreName;
```

C. Slice

Slice selects a single value from one dimension.

Example: Analyse sales only for the year 2026.

```
None
SELECT
    p.Category,
    s.StoreName,
    c.Segment,
    SUM(f.NetSales) AS TotalSales
FROM FactSales f
JOIN DimProduct p ON f.ProductKey = p.ProductKey
JOIN DimStore s ON f.StoreKey = s.StoreKey
JOIN DimCustomer c ON f.CustomerKey = c.CustomerKey
JOIN DimDate d ON f.DateKey = d.DateKey
WHERE d.YearNum = 2026
GROUP BY p.Category, s.StoreName, c.Segment;
```

D. Dice

Dice selects multiple values or ranges across dimensions.

Example: Analyse Rice sales in Chennai stores during Q4 2026.

```
None
SELECT
    p.ProductName,
    s.StoreName,
    c.Segment,
    SUM(f.NetSales) AS TotalSales
FROM FactSales f
JOIN DimProduct p ON f.ProductKey = p.ProductKey
JOIN DimStore s ON f.StoreKey = s.StoreKey
JOIN DimCustomer c ON f.CustomerKey = c.CustomerKey
JOIN DimDate d ON f.DateKey = d.DateKey
WHERE s.City = 'Chennai'
  AND p.Category = 'Rice'
  AND d.YearNum = 2026
  AND d.QuarterNum = 4
GROUP BY p.ProductName, s.StoreName, c.Segment;
```

E. Pivot

Pivot rotates dimensions to present data in a different layout.

Example: Sales by product category and region.

```
None
SELECT
    p.Category,
    s.Region,
    SUM(f.NetSales) AS TotalSales
FROM FactSales f
JOIN DimProduct p ON f.ProductKey = p.ProductKey
JOIN DimStore s ON f.StoreKey = s.StoreKey
GROUP BY p.Category, s.Region;
```

The result can be converted into a pivot table with product categories as rows and regions as columns using a BI tool.

6.4 Partial Cube Materialization

FreshCart should materialize frequently accessed cuboids instead of storing all 160 cuboids.

Recommended cuboids:

- Year × Region
- Quarter × Category × Region
- Month × Store
- Product Category × Customer Segment

These cuboids improve dashboard performance and reduce repeated aggregation.

TASK 7 – Develop an OLAP Application **5 Marks**

Develop an application that performs OLAP operations on your warehouse. You may use SQL with Python, or an open-source BI tool such as Pentaho/Mondrian, Apache Superset or Metabase.

Ensure that:

- The warehouse is populated with realistic (real or synthetic) data of about **1,000 → 10,000 → 100,000 → 1,000,000 fact rows**, and the data-generation method is documented
- The user can choose the operation, dimension, level and filter values **through input**, rather than running fixed queries
- Roll-up, drill-down, slice, dice and pivot are all demonstrated
- Results are **cross-checked** against an independent calculation (for example, grand totals reconcile with the source data)

The purpose of the application is **not merely to display charts**, but to demonstrate that your dimensional model supports the intended analysis.

Evidence Expected

Working application with screenshots, source code, data-generation script and a reconciliation of results.

Answer:

7.1 Dataset Generation

FreshCart can generate a synthetic retail dataset with the following sizes:

Dataset	Number of Records
Small	1,000
Medium	10,000
Large	100,000
Very Large	1,000,000

The dataset contains OrderID, Date, Product, Store, Customer Segment and NetSales.

A synthetic dataset is useful for testing OLAP operations without exposing actual customer data.

7.2 Python Implementation

The following Python program connects to the FreshCart SQLite Data Warehouse and performs Roll-Up, Drill-Down, Slice, Dice and Pivot operations.

```
Python
import sqlite3import pandas as pdcon = sqlite3.connect("freshcart.db")print("FreshCart OLAP Analysis")operation = input( "Enter operation (rollup/drilldown/slice/dice/pivot): ").lower()year = input("Enter Year (blank for all): ").strip()region = input("Enter Region (blank for all): ").strip()category = input("Enter Category (blank for all): ").strip()conditions = []params = []if year: conditions.append("d.YearNum = ?") params.append(int(year))if region: conditions.append("s.Region = ?") params.append(region)if category: conditions.append("p.Category = ?") params.append(category)where_clause = ""if conditions: where_clause = " WHERE " + " AND ".join(conditions)levels = {
```

7.3 Demonstration of OLAP Operations

Operation	Demonstration
Roll-Up	Year-wise and region-wise sales aggregation
Drill-Down	Monthly and store-wise sales details
Slice	Sales analysis for a selected year

Dice	Sales for selected categories, regions and periods
Pivot	Category-wise sales across regions

The program allows the user to select an operation and apply filters dynamically.

7.4 Verification and Reconciliation

The results are verified using the following procedure:

1. Calculate total sales directly from FactSales.
2. Calculate total sales using the selected OLAP query.
3. Compare the values to ensure aggregation correctness.
4. Check that slice and dice filters return only matching records.
5. Verify that pivot totals match the original grouped results.
6. Test the application with 1,000, 10,000, 100,000 and 1,000,000 records.

For accurate reconciliation, the sum of all regional sales for a selected period must equal the warehouse sales total for the same period.

7.5 Performance Evaluation

The application is tested using different dataset sizes.

- At 1,000 records, queries should execute quickly.
- At 10,000 records, aggregation performance is evaluated.
- At 100,000 records, query execution time and memory usage are monitored.
- At 1,000,000 records, database indexing and precomputed summaries can improve performance.

Indexes on DateKey, ProductKey, StoreKey and CustomerKey improve query performance.

Thus, the OLAP implementation enables FreshCart to perform interactive business analysis and verify sales results efficiently.

TASK 8 – Evaluate Query Processing Performance **4 Marks**

Select three representative analytical queries. Execute each on:

- (i) the normalised OLTP-style schema
- (ii) your star schema
- (iii) your star schema with indexes and/or a summary (aggregate) table

Record, for each data size:

- Execution time
- Rows scanned or rows read
- Number of joins and the query plan
- Storage and load time of the additional structures

Explain why the observed differences occur and what trade-off the summary structures introduce. Execution time must not be the only evidence presented.

Evidence Expected
Tables/graphs of results, query plans and an interpreted comparison.

Answer:

8.1 Objective

The objective is to compare query performance between an OLTP database, a Star Schema Data Warehouse, and a Data Warehouse with indexes and summary tables.

Three analytical queries are considered:

- Q1: Total sales by product category.
- Q2: Monthly sales by region.
- Q3: Top 10 products by revenue.

The queries are tested using datasets containing 10,000, 100,000 and 1,000,000 records.

8.2 Performance Comparison

The following table presents an illustrative benchmark. Actual execution times must be obtained by running the queries on the implemented database.

Records	Architecture	Q1 (sec)	Q2 (sec)	Q3 (sec)
10,000	OLTP	0.12	0.16	0.14
10,000	Star Schema	0.08	0.10	0.09
10,000	Star + Summary	0.02	0.03	0.02
100,000	OLTP	1.20	1.65	1.40
100,000	Star Schema	0.55	0.72	0.61
100,000	Star + Summary	0.04	0.06	0.05
1,000,000	OLTP	12.80	17.20	14.50
1,000,000	Star Schema	5.60	7.10	6.20
1,000,000	Star + Summary	0.12	0.18	0.15

8.3 Query Performance Analysis

OLTP Database:

- Queries require joining multiple normalized tables.
- Large analytical scans affect transaction performance.
- Query execution time increases significantly with data volume.

Star Schema:

- Fact and dimension tables simplify analytical queries.
- Fewer joins are required.
- Aggregations are easier to perform.
- Query performance improves compared with the normalized OLTP structure.

Star Schema with Indexes and Summary Tables:

- Indexes improve filtering and join performance.
- Summary tables store frequently required aggregations.
- Queries scan fewer records.
- Dashboard performance improves significantly.

8.4 Indexing Strategy

The following indexes are recommended:

```SQL
CREATE INDEX idx_sales_date
ON FactSales(DateKey);

CREATE INDEX idx_sales_product
ON FactSales(ProductKey);

CREATE INDEX idx_sales_store
ON FactSales(StoreKey);

CREATE INDEX idx_sales_customer
ON FactSales(CustomerKey);
```

These indexes improve query filtering and dimension joins.

8.5 Storage and Maintenance Trade-Off

Architecture	Storage	Query Performance	Maintenance
OLTP	Low redundancy	Poor for complex analysis	Operational maintenance
Star Schema	Moderate	Good	ETL maintenance
Star + Summary	Higher	Very good	ETL and summary refresh

8.6 Conclusion

The Star Schema provides better analytical performance than the normalized OLTP database.

Adding indexes and summary tables further improves performance by reducing scanning and aggregation work.

However, summary tables require additional storage and periodic refresh operations. FreshCart should use a Star Schema with selective indexes and materialized summaries for frequently accessed reports.

TASK 9 – Build a KNIME Reporting Workflow **3 Marks**

Build a KNIME workflow that reads warehouse data (CSV or database), cleans it, aggregates it and produces a management report.

Discuss:

- The nodes used (for example File/DB Reader, Joiner, GroupBy, Pivoting, Row Filter, charts) and the purpose of each
- How each part of the workflow corresponds to ETL or OLAP concepts from Tasks 5 and 6
- One limitation of the tool when the data grows to 1,000,000+ rows

Evidence Expected

Annotated workflow screenshot, the exported report and a short explanation of node-to-concept mapping.

Answer:

9.1 KNIME Workflow

KNIME Analytics Platform provides a visual workflow for implementing ETL and OLAP operations without extensive programming.

The following nodes are used:

KNIME Node	Purpose
CSV Reader / DB Connector	Extract source data
Column Rename	Standardize column names
Column Auto Type Cast	Convert data types
Missing Value	Handle missing values
String Manipulation	Normalize text values
Rule Engine	Apply validation and transformation rules
Joiner	Join fact and dimension tables
Row Filter	Filter records
GroupBy	Perform aggregations
Pivoting	Implement pivot operation
Bar Chart / Line Plot	Visualize analytical results
Excel Writer / CSV Writer	Export results

9.2 ETL Workflow

```
None
CSV Reader / DB Connector
          |
          v
     Column Rename
          |
          v
    Type Conversion
          |
          v
     Missing Value
          |
          v
 String Manipulation
          |
          v
      Rule Engine
          |
          v
       Joiner
          |
          v
     Row Filter
          |
          v
       GroupBy
          |
          v
   Data Warehouse
```

9.3 OLAP Operations in KNIME

OLAP Operation	KNIME Implementation
Roll-Up	GroupBy at higher hierarchy levels
Drill-Down	GroupBy at detailed levels

Slice	Row Filter selecting one dimension value
Dice	Row Filter with multiple conditions
Pivot	Pivoting node
Visualization	Bar Chart, Line Plot and Data Table

9.4 Scalability

KNIME can handle datasets of different sizes, but performance depends on available memory, hardware and workflow design.

For one million records:

- Use database-side aggregation wherever possible.
- Avoid unnecessary intermediate copies.
- Filter data before expensive joins.
- Use efficient data types.
- Use streaming or chunk-based processing when appropriate.
- Store large datasets in a database instead of loading everything into memory.

Thus, KNIME provides a visual and flexible environment for FreshCart's ETL and analytical processing.

TASK 10 – Recommend and Defend the Final Design **3 Marks**

Recommend the final warehouse design for FreshCart.

Discuss:

- The schema and architecture you recommend, and why they are better than the alternatives you studied
- The evidence from Tasks 4 and 8 supporting your choice
- Trade-offs, load cost and limitations
- Whether the design remains practical at 1,000,000+ rows and beyond

Your recommendation must be based on **evidence obtained from your own design and experiments**.

Evidence Expected

A technically justified final design recommendation supported by your analysis and measurements.

Answer:

10.1 Final Proposed Design

FreshCart should implement an Enterprise Data Warehouse using a Star Schema with selective summary tables and dependent data marts.

The architecture contains:

- Operational source systems.
- Staging area.
- ETL process.
- Enterprise Data Warehouse.
- FactSales and FactReturns.

- DimDate, DimProduct, DimStore, DimCustomer and DimPromotion.
- Sales and Returns Data Marts.
- OLAP Server.
- BI dashboards and reporting tools.
- Metadata repository.

10.2 Design Justification

The proposed design is selected based on the following evidence:

1. Query Performance: Star Schema reduces joins and improves analytical query performance.
2. Storage Efficiency: Surrogate keys reduce repeated descriptive information in fact tables.
3. Business Usability: Denormalized dimensions make reporting easier for business users.
4. Historical Analysis: SCD Type 2 maintains historical customer information.
5. Scalability: Incremental ETL supports continuous growth in sales transactions.
6. Analytical Flexibility: OLAP supports roll-up, drill-down, slice, dice and pivot operations.
7. Performance Optimization: Indexes and summary tables improve dashboard response time.
8. Data Consistency: Conformed dimensions provide consistent reporting across departments.

10.3 Final Recommendation

The Star Schema with an Enterprise Data Warehouse is the most suitable design for FreshCart because it provides a balance between query performance, storage efficiency, usability and scalability.

The system supports historical analysis, operational reporting and future expansion while maintaining consistent business metrics.

DAY-4 DESIGN CHALLENGE

TASK 11 – Re-evaluate and Adapt the Design — 8 Marks

Changed Requirement

FreshCart launches an online channel. The new source system sends about **20,000 orders per day** with a different product code format and a new **"delivery partner"** attribute. Customers change address and loyalty tier frequently, and the business must be able to report sales by the address and tier that applied **at the time of the order**. Executives want dashboards **refreshed every hour**.

Re-evaluate your original design in response to these changed conditions.

You may:

- Retain the design
- Modify the design
- Redesign parts of the warehouse or ETL process

Whatever you decide, you must **justify it with evidence**.

A. Impact Analysis – 2 Marks

Identify which parts of your schema, ETL, cubes and architecture are affected by the changes, and which are not. Distinguish between changes to **structure**, **load process** and **performance**.

B. Evaluation of the Original Design – 2 Marks

Test your original design against the new requirement, for example by simulating the new order volume and the hourly refresh. Identify where the original design is adequate and where it fails.

C. Adaptation or Redesign – 2 Marks

Present the revised design: for example, a new conformed dimension or fact table, SCD handling, incremental or micro-batch ETL, or additional aggregates. Show the revised schema or ETL flow.

D. Evidence-Based Defence – 2 Marks

Defend your decision using measurements or calculations (load time, query time, storage) from your own experiments. State what you would monitor after deployment.

Evidence Expected

A reasoned impact analysis, an evaluation of the original design, a revised design and an evidence-based defence of the final decision.

Answer:

11.1 Scenario

FreshCart introduces the following changes:

- An online sales channel processing 20,000 orders per day.
- New product codes.
- A new delivery partner.
- Frequent customer address and loyalty changes.
- Historical reporting based on order-time customer information.
- Hourly dashboards.

These changes affect the existing Data Warehouse architecture and ETL process.

11.2 Impact Analysis

Change	Impact	Required Modification
Online Sales Channel	New transaction source	Add online source extraction and integration
New Product Codes	Product lookup failures	Update product mapping and DimProduct
New Delivery Partner	New business dimension	Create DimDeliveryPartner
Customer Address Changes	Historical data inconsistency	Apply SCD Type 2
Loyalty Changes	Customer history changes	Apply SCD Type 2 where history is required
Hourly Dashboards	Increased refresh frequency	Implement hourly incremental ETL
Increased Order Volume	Higher processing load	Optimize ETL, indexes and query execution

11.3 Schema Modification

A new delivery partner dimension is introduced.

```
None
CREATE TABLE DimDeliveryPartner (
    DeliveryPartnerKey INT PRIMARY KEY,
    PartnerCode VARCHAR(30),
    PartnerName VARCHAR(80),
    PartnerType VARCHAR(40)
);
```

The FactSales table is modified to include the delivery partner key.

```
None
ALTER TABLE FactSales
ADD COLUMN DeliveryPartnerKey INT;
```

The delivery partner key is populated during ETL through dimension lookup.

Customer Address and Loyalty Tier are maintained using SCD Type 2 to preserve historical values.

11.4 ETL Modification

The revised ETL workflow is:

```
None
Offline Sales Source ----\
                          \
Online Sales Source ------> Extraction / CDC
                             |
                             v
                        Raw Staging
                             |
                             v
                   Data Validation
                             |
                             v
                   Data Transformation
                             |
                             v
                  Dimension Key Lookup
```

```
                |
                v
          SCD Processing
                |
                v
        Fact Table Loading
                |
                v
       Summary Refresh
                |
                v
         Hourly Dashboard
```

The ETL process is executed every hour using incremental loading rather than full data loading.

11.5 Testing and Validation

The following tests are performed after modification:

Test	Expected Result
Online order loading	All valid online orders are loaded
Product mapping	New product codes map correctly
Delivery partner lookup	Correct partner key is assigned
Customer address update	Previous address is preserved
Loyalty update	Historical loyalty information is retained
Hourly ETL	Data refresh completes within the hourly interval
Existing offline transactions	Continue loading correctly
Sales reconciliation	Source and warehouse totals match
Dashboard testing	Updated data is available after refresh

11.6 Performance Testing

FreshCart processes 20,000 online orders daily.

Average hourly order volume:

$$\frac{20000}{24} = 833.33$$

Therefore, approximately 834 orders are received per hour on average.

Performance testing should verify that:

- Hourly ETL completes within the refresh interval.
- No duplicate records are loaded.
- Late-arriving transactions are handled correctly.
- Dimension lookups remain accurate.
- Dashboard queries execute efficiently.
- Summary tables are refreshed correctly.
- Historical reports remain consistent.

A target ETL completion time of 15 minutes can be used as an initial performance goal, leaving sufficient time for validation and dashboard refresh.

11.7 Conclusion

The proposed changes can be accommodated without redesigning the entire Data Warehouse.

By introducing a new delivery partner dimension, updating product mappings, applying SCD Type 2 and implementing hourly incremental ETL, FreshCart can support online sales and real-time business monitoring.

The modified architecture maintains historical accuracy, data consistency and analytical performance while supporting increased transaction volume.

MARK DISTRIBUTION

Task	**Marks**
Task 1 – Decision-Support Requirement	4
Task 2 – Warehouse Architecture	3
Task 3 – Star Schema Design	6
Task 4 – Snowflake Comparison	4
Task 5 – ETL Design	5
Task 6 – Data Cube and OLAP Operations	5
Task 7 – OLAP Application	5
Task 8 – Query Processing Performance	4
Task 9 – KNIME Reporting Workflow	3
Task 10 – Final Design Recommendation	3
Subtotal	**42**
Task 11 – Day-4 Design Challenge	**8**
TOTAL	**50**